

AI Security Learning Program

Week 2 — Hands-On Lab Report

Core Security Concepts Applied to AI Systems

Topics Covered: Authentication • Authorization • RBAC • ABAC • MFA • IAM • Audit Logs • Encryption • Data Privacy •
Secure APIs

1. Executive Summary

This report documents the Week 2 hands-on lab for the AI Security learning roadmap, covering core cybersecurity foundations and their direct application to AI system design. The lab consisted of three parts: (1) identifying and fixing vulnerabilities in a deliberately insecure AI-assistant API, (2) analyzing a real-world breach (the 2025 McDonald's "Olivia" hiring chatbot incident), and (3) evaluating six realistic AI system design scenarios for security soundness.

The central finding across all three parts is consistent: AI-specific security incidents are rarely caused by the AI model itself behaving unexpectedly. They are overwhelmingly caused by traditional, well-understood security gaps — broken authorization, weak credentials, missing audit trails, and over-permissioned access — that happen to sit underneath an AI interface.

2. Objectives

- Apply core IAM concepts (Authentication, Authorization, RBAC, ABAC, MFA) to a working API.
- Identify and remediate a Broken Object Level Authorization (BOLA) vulnerability — the most common real-world API flaw.
- Connect a documented AI security breach to specific, named security controls.
- Practice evaluating AI system designs for authorization, audit, and human-oversight soundness.

3. Part 1 — Secure API Development

A simplified Node.js/Express API for an AI HR assistant was reviewed. It contained three intentional vulnerabilities modeled directly on real incidents discussed earlier in the program.

3.1 Vulnerabilities Identified and Fixed

#	Vulnerability	Concept	Fix Applied
1	Hardcoded JWT signing secret in source code	Secrets Management	Moved to environment variable (process.env.JWT_SECRET)
2	Plaintext password storage, weak password ("123456")	Authentication	bcrypt password hashing + minimum password policy
3	No ownership check on GET /employee/:id (BOLA)	Authorization	Added object-level (owner/manager/admin) and property-level (salary field) checks

3.2 Supporting Controls Added

- Rate limiting on the /login endpoint (5 requests / 15 min) to mitigate brute-force and MFA-fatigue-style abuse.
- Reduced JWT expiry from 24h to 1h, limiting the exposure window of a stolen token.
- Audit log middleware recording actor, action, target, outcome, and timestamp on every request.

3.3 Residual Gap (Flagged for Awareness)

The fixed version does not yet include `on_behalf_of` or `triggering_prompt` fields in the audit log. These become necessary once an AI agent — rather than a human directly — calls this API, to preserve accountability back to the requesting human and to support forensic review of why an action occurred.

4. Part 2 — Breach Case Study: McDonald's "Olivia" (June 2025)

The McDonald's AI-powered hiring assistant, operated by Paradox.ai, exposed personal information belonging to over 64 million applicants, including private chat transcripts. Two failures combined: a BOLA vulnerability allowing retrieval of arbitrary applicant records, and a test admin account left active with the password "123456."

4.1 Analysis Findings

Question	Finding
Which lab vulnerability matches this breach?	Fix 3 (BOLA) is the direct match; Fix 2 (weak credentials) also matches the exposed admin password exactly.
Was this "AI going rogue"?	No. The chatbot performed as designed. The failure was a conventional API authorization gap, unrelated to any AI-specific behavior.
Highest-priority control to check pre-launch?	Authorization / BOLA testing — the single most common API vulnerability class (over 40% of reported API vulnerabilities in 2026 data).
Which of the 4 A's of IAM does the forgotten admin account belong to?	Administration — a lifecycle failure to deprovision a test account, not a design flaw in authentication or authorization logic.

5. Part 3 — Scenario-Based Security Evaluation

Six realistic AI system design decisions were evaluated for security soundness.

Scenario	Verdict	Governing Principle
AI agent uses its own long-lived service key on behalf of any user	Not Secure	Use intersection, not union, of agent + user permissions
RAG chatbot retrieves all data, filters via system prompt instruction	Not Secure	Filter at retrieval/data layer, not via prompt
New integration granted "Allow All" scope, to be narrowed later	Not Secure	Principle of Least Privilege from day one
Audit log entry omits <code>on_behalf_of</code> field	Not Secure	Accountability requires linking agent action to human requester
Banking chatbot drafts a transfer, requires human confirmation before execution	Secure	Human-in-the-loop for irreversible actions

Scenario	Verdict	Governing Principle
Push-notification MFA with no send-rate limit	Not Secure	Rate limiting prevents MFA-fatigue / prompt-bombing attacks

6. Key Learnings — Concept Map

Topic	One-Line Takeaway
Authentication	Verifies identity; short-lived tokens reduce blast radius versus static long-lived keys.
Authorization	Determines allowed actions; must be enforced at the data/API layer, never via prompt instruction alone.
RBAC	Coarse-grained, role-based access; simple to manage but prone to role explosion at scale.
ABAC	Fine-grained, attribute-based access; resolves RBAC's inability to express record-level rules.
MFA	Strengthens authentication with independent factors; modern attacks bypass it via token theft rather than weak passwords.
IAM (4 A's)	Administration, Authentication, Authorization, and Audit together form the full identity lifecycle.
Audit Logs	Enable detection and forensics; AI systems additionally require <code>on_behalf_of</code> and <code>triggering_prompt</code> fields.
Encryption	Protects data from unauthorized parties; does not prevent an authorized model from leaking data it legitimately processed.
Data Privacy	Governs whether data should be collected/used at all — a distinct question from security.
Secure APIs	The enforcement layer where every IAM control is physically implemented; BOLA is the most common real-world failure.

7. Self-Assessment Checklist

The following questions were used to verify conceptual understanding beyond rote definitions:

- Why is the intersection of agent and user permissions safer than their union?
- Why does encryption not solve the RAG data-leakage problem?
- What is the difference between object-level and property-level authorization?
- Why did the McDonald's breach have nothing to do with "AI going rogue"?
- What two fields make an audit log AI-aware rather than only security-aware?

8. Conclusion and Next Steps

Week 2 established that AI security is built on the same foundational IAM and API security principles as traditional application security. The recurring theme across all real-world cases examined — Vercel/Context.ai, the Obsidian Salesforce incident, and the McDonald's Olivia breach — is that a system prompt or model

instruction is a request, while a database- or API-level control is a guarantee. Security must be enforced at the level of guarantees.

This foundation directly prepares for Phase 3 of the roadmap: the OWASP Top 10 for LLM Applications, where AI-specific risks such as Prompt Injection, Excessive Agency, and Sensitive Information Disclosure will be shown to be extensions of the same authorization and data-handling principles covered this week.

— *End of Report* —